more difficult for the algorithm to memorize. This ought to enforce some amount of actual learning, but as you saw, we're still experiencing difficulty with learning this variant; we're getting very noisy loss plots. To help address this, we'll add another dimension to the updating rule that will smooth out the value updates.

3.4.1. Learning instability

One potential problem that DeepMind identified when they published their deep Q-network paper was that if you keep updating the Q-network's parameters after each move, you might cause instabilities to arise. The idea is that since the rewards may be sparse (we only give a significant reward upon winning or losing the game), updating on every single step, where most steps don't get any significant reward, may cause the algorithm to start behaving erratically.

For example, the Q-network might predict a high value for the "up" action in some state; if it moves up and by chance lands on the goal and wins, we update the Q-network to reflect the fact that it was rewarded +10. The next game, however, it thinks "up" is a really fantastic move and predicts a high Q value, but then it moves up and gets a –10 reward, so we update and now it thinks "up" is not so great after all. Then, a few games later moving up leads to winning again. You can see how this might lead to a kind of oscillatory behavior, where the predicted Q value never settles on a reasonable value but just keeps getting jerked around. This is very similar to the catastrophic forgetting problem.

This is not just a theoretical issue—it's something that DeepMind observed in their own training. The solution they devised is to duplicate the Q-network into two copies, each with its own model parameters: the "regular" Q-network and a copy called the *target network* (symbolically denoted $\hat{Q}$-network, read "Q hat"). The target network is identical to the Q-network at the beginning, before any training, but its own parameters lag behind the regular Q-network in terms of how they're updated.